Auditoría de usuarios, autenticación, membresías y permisos

Fecha: **22 de octubre de 2026**, America/Costa_Rica. Instrucciones: Agents.md. Alcance: código del árbol de trabajo actual, incluidas modificaciones preexistentes; no solo la versión de Git. Esta entrega documenta hallazgos y reproducciones; **no modifica ni remedia el código de producción**.

1. Resumen ejecutivo

El módulo no debe considerarse seguro para cuentas compartidas entre clientes hasta corregir U01. Un administrador de una organización puede vincular una cuenta existente con solo conocer su correo, modificar su contraseña global y autenticarse como esa persona en otra organización. La misma cadena permite acceder a plataforma si la cuenta afectada tiene `esSuperAdministrador=true`.

Se identifican **18 hallazgos: 1 P0, 12 P1 y 5 P2**. Los controles de autorización directa de usuarios y de selección de finca funcionan en las pruebas; el problema crítico aparece al combinar operaciones autorizadas sobre una identidad global.

Otros riesgos silenciosos:

- Cambiar o recuperar una contraseña no invalida los JWT anteriores, de ocho horas; el selector de organización permite renovarlos sin conocer la nueva contraseña.
- La última membresía administradora puede eliminarse, desactivarse o degradarse.
- La auditoría deja contraseñas de confirmación y ciertos tokens sin ocultar.
- El logout limpia únicamente el contexto de la finca actual. Otros contextos y sus colas permanecen.
- Un token de recuperación puede consumirse dos veces concurrentemente.
- La descarga de imágenes de dron verifica organización, pero no permiso del módulo ni finca del archivo.
- Las altas de identidades nuevas fallan por una condición incorrecta que las considera inactivas.

Ataques probados y resultado

Prueba local	Resultado observado
Administrador A vincula correo de B, cambia contraseña y entra como B	**Reproducido**, incluida plataforma para una víctima superadministradora
Encargado, Trabajador, Veterinario, Contador o Consulta intentan asignar Administrador	**403** en las cinco pruebas, incluso con rol Administrador dentro del JWT
Admin A consulta directamente identidad exclusiva de B, antes de vincularla	**404**
Forzar organización ajena en selector o en JWT firmado de pruebas	**403** por ausencia de membresía
Forzar finca de otra organización usando `X-Finca-Id`	**403**
Enviar `esSuperAdministrador` y `organizacionId` en edición autorizada	Ignorados; no cambian esos campos
Eliminar, desactivar o degradar al único administrador	**200**; se pierde el administrador
Reutilizar JWT después de cambiar o recuperar contraseña	**200**
Usar JWT anterior al cambio de contraseña para seleccionar la misma organización	**200**, nuevo JWT con expiración posterior
Reutilizar JWT después de baja de membresía o desactivación global	**403**; rol nuevo también se aplica inmediatamente
50 fallos seguidos, misma IP y correo	**5 × 401 y 45 × 429**
Variar 50 correos desde una IP / 50 IP para un correo	El limitador admite los 50 intentos en cada caso
Enumerar cuenta inactiva con contraseña incorrecta	**403** distinto del **401** de correo inexistente
JWT `alg:none` sin firma	**401**

Prueba local	Resultado observado
JWT con encabezado `alg:none`, sin `exp`, pero con HMAC válido de pruebas	**200**: defecto de validación, **no** bypass sin conocer el secreto
Reset secuencial con token sustituido o consumido	**400**
Dos resets concurrentes con el mismo token vigente	**200 + 200**, contraseña final dependiente del orden
Dos vinculaciones simultáneas en el último cupo Esencial	**201 + 422**, tres miembros activos finales
Descarga de dron por Consulta	El router autoriza la descarga simulada aunque `drone.ver=false`
Sincronización offline con respuesta 403	Conserva operación y mensaje; no actualiza caché como éxito

Evidencia y límites de las pruebas

Reproductor: backend/documentation/auditoria-usuarios.repro.cjs.

```
cd backend
node documentation/auditoria-usuarios.repro.cjs
npm.cmd test
npm.cmd audit --json
npm.cmd audit --omit=dev --json
cd ../frontend
npm.cmd run test:tareas
```

Resultados: **36 comprobaciones específicas**, **307 pruebas existentes del backend**, **4 pruebas existentes de tareas offline**, todas completadas. Las comprobaciones específicas afirman el comportamiento actual, incluidos los defectos: que pasen **no significa que se hayan corregido**. Tras remediar, las expectativas deberán invertirse.

El reproductor levanta HTTP exclusivamente en `127.0.0.1`, ejecuta rutas, middleware, controladores, bcrypt, servicios de permisos/cuotas y selección de finca reales. Sustituye la persistencia por adaptadores en memoria, el correo por capturas y la descarga por un marcador sin leer archivos. Para probar autorizaciones de otros módulos, sustituye controladores de negocio y guardas de plan, manteniendo las guardas de rol reales. Ejecuta un hook real de aislamiento Mongoose sin consultar MongoDB.

No se usaron cuentas, correos, bases de datos ni endpoints de producción. Las pruebas de concurrencia demuestran el flujo con persistencia simulada; quedan pendientes integración MongoDB con índices reales, pruebas de varias instancias, navegador/IndexedDB real y configuración TLS del despliegue. Las comprobaciones de JSX son inspecciones reproducibles del código y peticiones a sus rutas, no pruebas visuales de navegador.

2. Catálogo de procesos y mapa real

Proceso	Descripción en código	Backend	Frontend	Madurez
Alta inicial SaaS	Superadministrador crea organización, finca, cuenta/membresía y catálogos en transacción	`adminOrganizacionRoutes`, `aprovisionamientoOrganizacion-service`	Administración SaaS	Implementado; aceptación y correo mejorables
Registro público	No hay endpoint público de registro	POST usuarios exige administrador	`Crearusuario.js` legado no está montado en `App.js`	No existe como flujo público
Alta de usuario nuevo	Pretende crear identidad bcrypt y	POST `/api/usuarios`	`pages/Usuarios.js`	**Roto, U02**

Proceso	Descripción en código	Backend	Frontend	Madurez
	membresía			
Vincular usuario existente	Busca correo global y crea membresía inmediatamente	Mismo POST	Formulario de usuarios	**Crítico, U01**, sin aceptación
Login	Normaliza correo, valida clave, elige membresía activa, emite JWT	POST `/api/usuarios/login`	`IniciarSesion.js`	Parcial, U05/U06
Perfil propio	Solo consulta, elimina campos privados	GET `/api/usuarios/perfil`	Validación inicial de `App.js`	Parcial, U16
Organizaciones propias	Lista membresías activas de organizaciones activas	GET `/mis-organizaciones`	Selector	Implementado
Cambiar organización	Comprueba membresía y emite token ligado a organización	POST `/cambiar-organizacion-activa`	`App.js`	Implementado; requiere contexto actual válido
Acceso a finca	`accesoTodasFincas` o lista de IDs; consulta organización y estado	`auth`, `accesoFinca-service` en cada petición	Selector + `X-Finca-Id`	Implementado
Edición de usuario	Modifica identidad global y membresía local, en dos guardados	PUT `/api/usuarios/:id`	Gestión usuarios	**Crítico, U01**
Baja/desactivación	Borra o inactiva membresía; conserva identidad global	DELETE `/:id`, PATCH `/:id/estado`	Gestión usuarios	Parcial, U03; sin reasignación operativa
Recuperación	Token aleatorio, hash SHA-256, 30 minutos y correo	`/api/auth/forgot-password`; alias `/api/usuarios/recuperar-contrasena`	`OlvideContrasena.js`	Parcial, U04/U05/U11
Restablecimiento	Busca token vigente, guarda bcrypt y borra token	`/api/auth/reset-password`; alias `/api/usuarios/restablecer-contrasena`	`RestablecerContrasena.js`	Parcial, U08
Invitación inicial	Para cuenta nueva de administrador usa el mismo reset, TTL 24 horas	Aprovisionamiento y servicio de correo	Misma pantalla de reset	Sin entidad ni estado de invitación
Cupos	Cuenta membresías activas por organización y reserva serializada	`plan-service`, `ReservaCuotaPlan`	`PlanContext`, usuarios	Correcto en simulación concurrente

Proceso	Descripción en código	Backend	Frontend	Madurez
Plataforma	Relee bandera global y estado; no confía en rol del JWT	`authPlataforma`, `/api/admin/organizaciones`	Botón para superadministrador	Guarda correcta; afectada por U01
Logout/offline	Token local + IndexedDB particionado; borrado de contexto	No hay endpoint de logout/revocación	`App`, `offlineStorage`, `offlineSync`	Parcial, U12/U13
Auditoría administrativa	Registra mutaciones al terminar respuesta	`auditoriaPeticiones`, `Auditoria`	Vista en Usuarios, Pro/Premium	Parcial, U09/U10

Modelos: `Usuario` es global; `Membresia` tiene índice único `(organizacionId, usuario)` y rol por organización. `Organizacion` guarda estado, plan y finca principal. No existe modelo `Invitacion`, tabla de sesiones ni modelo separado de recuperación: los cuatro campos de reset están en `Usuario`. `Auditoria` tiene aislamiento por organización y `EventoFacturacion` registra procesamiento de eventos Stripe.

No hay rutas `/api/membresias/*` ni `/api/invitaciones/*`. Las operaciones de membresía se realizan bajo `/api/usuarios`. `usuarioRoutes.js` y `usuarioController.js` son alias de los archivos principales, no implementaciones alternativas.

3. Sesión, contraseñas y tokens

- **Hash:** bcrypt instalado **6.0.0**, costo **10**. No se usan jsonwebtoken, passport ni argon2. JWT implementado con `crypto.createHmac('sha256', ...)` de Node.
- **Política:** reset exige ocho caracteres; edición administrativa admite `x` y no exige contraseña actual. La rama de alta tampoco define longitud mínima, aunque hoy queda bloqueada por U02. No hay verificación de correo. El modelo tiene `lowercase:true`, `trim:true` e índice único de correo; controladores también normalizan.
- **Legado:** login acepta comparación de contraseña en texto plano si no reconoce prefijo bcrypt y migra a hash al acceder. No se consultó la base real, por lo que **no se afirma que existan contraseñas en texto plano actualmente**.
- **Exposición:** las respuestas principales de usuarios eliminan contraseña y campos de reset mediante `limpiarUsuario`/`CAMPOS_PRIVADOS`. El esquema no usa `select:false`; la protección depende del serializador. El test comprobó login y perfil sin hash. U09/U11 afectan registros y correos, no esa serialización.
- **Secreto:** solo `JWT_SECRET`, sin fallback; ausencia produce error. No hay validación de longitud/entropía al arrancar. No se inspeccionaron secretos reales.
- **Expiración:** ocho horas, sin refresh token, rotación, identificador de sesión, lista de revocación ni versión de contraseña. Cambiar organización emite otro token por ocho horas; el anterior conserva validez para su organización mientras exista membresía.
- **Validación JWT:** la firma siempre se calcula con SHA-256, pero no se interpreta ni valida el encabezado. `exp` se comprueba únicamente si es truthy; faltan validaciones de tipo, `iss`, `aud` y `nbf`. Comparación de firma con `!==`, no `timingSafeEqual`. U07 requiere firma válida para demostrar aceptación; no se consiguió autenticación con firma inexistente o expirada.
- **Navegador:** `ganaderiaSesion` en localStorage contiene token e identidad/contexto. Peticiones usan Authorization Bearer; no se usan cookies de autenticación. La exposición a código del mismo origen es inherente; **no se ha demostrado una XSS del sitio en esta auditoría**. CSRF basado en cookies no aplica al mecanismo observado; CORS no sustituye autorización.
- **Revocación efectiva:** cada petición relee usuario, membresía, rol y finca. Baja o inactividad devuelve 403; cambio de rol aplica inmediatamente aunque el JWT diga Administrador. Cambio/reset de contraseña no afecta al JWT. Además, un token aún válido puede llamar `cambiar-organizacion-activa` incluso con la misma organización y obtener otro de ocho horas sin contraseña. Se reprodujo después del cambio de clave, comparando ambas expiraciones. Por tanto, ocho horas **no es un límite total de persistencia del atacante** si renueva antes de expirar. Logout solo borra cliente, por lo que una copia externa del token tampoco se revoca.
- **Enumeración:** inexistente y contraseña incorrecta de activo devuelven 401 genérico; inactivo devuelve 403 antes de comparar clave. Medianas de cinco ejecuciones locales sin DB/red: inexistente **0,01 ms**, existente **57,80 ms** en la última corrida. Es evidencia de una ruta con costo distinto, no una medición de explotabilidad por Internet.
- **Rate limit:** cinco intentos por 15 minutos por combinación `IP:correo`, en un Map del proceso, sin bloqueo por cuenta o IP independiente. Variar una dimensión evita el límite agregado. No se configura `trust proxy`; detrás de un proxy, verificar la IP efectiva antes de aplicar cambios.

- **Estado:** organización Suspendida/Inactiva bloquea login y peticiones, no ofrece lectura. Plan Suspendido/Cancelado en organización Activa permite autenticación/perfil y consultas cuyas rutas no exigen feature vigente. Altas/reactivaciones validan plan vigente. No equivale a modo universal de solo lectura: algunas operaciones administrativas existentes no consultan el plan. Vigencia usa estado, no compara `fechaExpiracion` directamente; depende de la actualización de facturación/administración.

4. Recuperación e invitaciones

Control	Recuperación	Invitación inicial SaaS
Entropía	`randomBytes(32)`, 64 caracteres hex	Mismo generador
Persistencia	SHA-256 del token en Usuario	Mismos campos, cuenta nueva
TTL	30 minutos	24 horas
Nuevo token invalida anterior	Sí, reemplazo de campo	Reset posterior también reemplaza invitación
Uso único secuencial	Sí, segundo consumo 400	Usa mismo endpoint
Uso único concurrente	**No**, U08	Comparte defecto
Revoca otras sesiones	**No**, U04	No hay versión de sesión
Enlace por Host	No; `FRONTEND_URL`, fallback localhost	Igual
Escape HTML	**No**, nombre sin escape	Nombre y organización sin escape
Sin proveedor de correo	Log imprime mensaje con token; respuesta de recuperación puede aparentar éxito	Aprovisionamiento indica invitación no enviada; enlace de prueba solo fuera de production
Limitación	Tres solicitudes/30 minutos por IP+correo	No hay endpoint público de invitación; creación restringida a plataforma
Respuesta indistinguible	Existente/inexistente normalmente 200 genérico; existente espera envío y error de proveedor produce 500	No aplica al alta interna
Confirmación de identidad	Posesión del token permite cambiar contraseña; no requiere sesión de otro correo	No hay aceptación separada de membresía

El endpoint de reset no necesita correo, pero el rate limit lo incorpora a la clave. Se probaron diez intentos con token inválido cambiando ese campo irrelevante: los diez llegaron al controlador (**400**, no 429). No facilita adivinar un token de 256 bits por sí solo; sí anula el límite y permite presión sobre consultas/correo según la ruta.

Para usuarios existentes no hay invitación: `crearUsuario` y el aprovisionamiento los vinculan directamente. No existe comprobación de consentimiento del invitado ni estado Pendiente. La invitación para una nueva organización activa la membresía antes de establecer contraseña. Sus requisitos no deben documentarse como si existiera un flujo aceptar/rechazar.

5. Matriz real de roles y comparación de capas

Fuente: backend/config/permisosRoles.js. A=Administrador, E=Encargado, T=Trabajador, V=Veterinario, C=Contador, L=Consulta. La matriz expresa permiso base; plan, finca, asignación y estado pueden restringir adicionalmente.

Permiso	A	E	T	V	C	L
dashboard.ver	Sí	—	—	—	—	—
tareas.verTodas	Sí	Sí	—	—	—	—
tareas.gestionar	Sí	Sí	—	—	—	—
tareas.verAsignadas	Sí	Sí	Sí	Sí	Sí	Sí

Permiso	A	E	T	V	C	L
tareas.completarAsignadas	Sí	Sí	Sí	Sí	Sí	—
importar.gestionar	Sí	—	—	—	—	—
inventario.ver	Sí	Sí	Sí	Sí	—	Sí
inventario.gestionar	Sí	Sí	—	—	—	—
lotes.ver	Sí	Sí	Sí	Sí	—	Sí
lotes.gestionar	Sí	Sí	—	—	—	—
lotes.protocoloConfigurar	Sí	—	—	—	—	—
lotes.protocoloCrear	Sí	Sí	—	—	—	—
lotes.protocoloEjecutar	Sí	Sí	Sí	Sí	—	—
alimentacion.ver	Sí	Sí	Sí	Sí	—	Sí
alimentacion.gestionar	Sí	Sí	—	—	—	—
alimentacion.registrarSuministros	Sí	Sí	Sí	—	—	—
camadas.ver	Sí	Sí	—	Sí	—	Sí
camadas.gestionar	Sí	Sí	—	Sí	—	—
pesajes.ver	Sí	Sí	—	Sí	—	Sí
pesajes.gestionar	Sí	Sí	—	Sí	—	—
pesajes.eliminar	Sí	—	—	—	—	—
potreros.ver	Sí	Sí	Sí	—	—	Sí
potreros.gestionar	Sí	Sí	—	—	—	—
sanidad.ver	Sí	Sí	—	Sí	—	Sí
sanidad.gestionar	Sí	Sí	—	Sí	—	—
sanidad.eliminar	Sí	—	—	—	—	—
bitacora.ver	Sí	Sí	Sí	Sí	—	Sí
bitacora.crearNotas	Sí	Sí	—	Sí	—	—
reproduccion.ver	Sí	Sí	—	Sí	—	Sí
reproduccion.gestionar	Sí	Sí	—	Sí	—	—
reproduccion.iatfConfigurar	Sí	—	—	Sí	—	—
reproduccion.iatfCrear	Sí	—	—	Sí	—	—
reproduccion.iatfEjecutar	Sí	Sí	Sí	Sí	—	—
reproduccion.iatfDiagnosticar	Sí	—	—	Sí	—	—
reproduccion.protocoloPorcinoConfigurar	Sí	—	—	Sí	—	—
reproduccion.protocoloPorcinoCrear	Sí	—	—	Sí	—	—
reproduccion.protocoloPorcinoEjecutar	Sí	Sí	Sí	Sí	—	—
reproduccion.protocoloPorcinoDiagnosticar	Sí	—	—	Sí	—	—
reproduccion.eliminar	Sí	—	—	—	—	—
compras.ver	Sí	Sí	—	—	Sí	Sí

Permiso	A	E	T	V	C	L
compras.gestionar	Sí	Sí	—	—	—	—
compras.eliminar	Sí	—	—	—	—	—
ventas.ver	Sí	Sí	—	—	Sí	Sí
ventas.gestionar	Sí	Sí	—	—	—	—
ventas.eliminar	Sí	—	—	—	—	—
finanzas.ver	Sí	—	—	—	Sí	—
finanzas.gestionar	Sí	—	—	—	Sí	—
finanzas.administrarCatalogos	Sí	—	—	—	—	—
reportes.ver	Sí	—	—	—	Sí	Sí
reportes.configurar	Sí	—	—	—	—	—
drone.ver	Sí	Sí	—	—	—	—
drone.gestionar	Sí	Sí	—	—	—	—
drone.eliminar	Sí	—	—	—	—	—
usuarios.gestionar	Sí	—	—	—	—	—

`rolTienePermiso` permite todo a Administrador antes de buscar el nombre del permiso, incluso uno inexistente. Es una decisión de implementación, no una escalada demostrada. El frontend tiene tablas independientes de navegación, gestión y cuatro permisos por acción; no comparte la matriz completa.

Contraste con rutas y frontend

Grupo	Aplicación real del backend	Frontend / diferencia
Dashboard	No hay endpoint `dashboard.ver`; compone consultas de otros módulos	Solo A. Guarda de navegación, no permiso que proteja un endpoint único
Tareas	Autenticación en router; autorización y asignado dentro de `tareaController`, no ausente	A/E ven Todas; resto Mis tareas. Consulta no modifica
Inventario/genealogía	`inventario.ver/gestionar`; sanidad independiente en PATCH sanitario	Coincide para animales; **camadas heredan soloLectura de Inventario**
Camadas	`camadas.ver/gestionar`; borrar requiere `reproduccion.eliminar`	**Veterinario puede gestionar por API pero ve solo lectura en Inventario; Encargado ve Eliminar y obtiene 403; Trabajador dispara GET camadas 403 al elegir porcinos**
Lotes/protocolos de engorde	`lotes.ver/gestionar`; pesajes/tareas/potreros exigen permisos propios; protocolos usan permisos por acción	Guardas de protocolo específicas presentes; **Veterinario carga potreros y recibe 403 silenciado**
Alimentación	`alimentacion.ver/gestionar/registrarSuministros` según acción	Coincide con permisos separados de registrar y editar

Grupo	Aplicación real del backend	Frontend / diferencia
Pesajes	`.ver/.gestionar/.eliminar`, borrar solo A	Botón eliminar sí restringido a A; **GET potreros 403 de V se convierte en lista vacía**
Potreros/pastos/rotaciones	`potreros.ver/gestionar`	Navegación coherente
Sanidad/planes/tratamientos	`.ver/.gestionar`, eliminación de planes/registros solo A	**Planes muestra Eliminar a E/V usando soloLectura; API 403**
Bitácora animal	`.ver`, `.crearNotas`; editar/borrar `usuarios.gestionar`	Crear nota usa permiso por acción; eventos de camada usan `camadas.gestionar`
Reproducción	`.ver/.gestionar/.eliminar`	**Eliminar visible a E/V, API 403**
IATF	Configurar/crear/diagnosticar A/V; ejecutar A/E/T/V; lectura `.ver`	Pantalla de reproducción A/E/V/L; Trabajador opera desde tareas, no se debe exigirle acceso a toda la pantalla
Bandas porcinas	Crear/configurar A/V; ejecución A/E/T/V; diagnóstico adicional en servicio	**SoloLectura de reproducción habilita acciones de creación a E; GET asignables usa permiso de creación, devuelve 403 y rompe Promise.all de la carga**
Compras y ventas	`.ver/.gestionar/.eliminar`	Consultas y selectores diferidos evitan cargas de edición a C/L; **Eliminar visible a E, API 403**
Finanzas	`.ver/.gestionar`, catálogos administrativos solo A; destinos-resumen `reportes.ver`	Tablas de módulo y acción coherentes; destinos es recurso de reportes
Reportes	Router usa `reportes.ver`; configuración/metas/cierres exigen `.configurar`	A/C/L consultan; configuración A; features de plan adicionales
Dron	Rutas de conteos usan `.ver/.gestionar/.eliminar`	Navegación A/E; **ruta de imágenes no verifica esos permisos, U14**
Usuarios/auditoría/fincas/apariencia	Administración usa `usuarios.gestionar`; auditoría además feature Pro+	Usuarios solo A; consultas propias de perfil/fincas/apariencia accesibles a roles autenticados
Plan/facturación	`/plan/actual` para autenticados; facturación solo A; especie solo A	Mi plan solo A; `PlanContext` consulta el estado para todos, coherente con backend
Notificaciones/configuración email	Usuario/destinatario actual en controlador; email depende de feature y permisos financieros	Acceso general autenticado, sin permiso nominal

Grupo	Aplicación real del backend	Frontend / diferencia
		específico en matriz
Plataforma	`authPlataforma`, bandera global actual	Flag para mostrar acceso; la seguridad real se verifica en backend

U15 reproduce por HTTP las denegaciones de cinco eliminaciones, GET potreros/camadas y creación de banda; contrasta las condiciones JSX. Las diferencias adicionales de navegación/ocultación descritas son inspección de fuente. No se afirma haber recorrido visualmente todos los componentes.

6. Escalada y aislamiento entre organizaciones

U01 — P0: toma de cuenta global desde una organización

Código: usuario-controller.js, `crearUsuario` desde línea 151 y `actualizarUsuario` desde 212. La membresía limita qué usuario se edita, pero **el mismo administrador puede crear previamente esa membresía sin autorización del titular**. La identidad editada incluye correo, teléfono, nombre y contraseña globales.

Reproducción ejecutada en HTTP local, con IDs y cuentas ficticios:

```
POST /api/usuarios
Authorization: Bearer <token-admin-A>
Content-Type: application/json

{"correo":"victima-b@example.test","rol":"Consulta"}
```

Respuesta 201 con ID global de la víctima, sin invitación ni correo enviado. Luego:

```
PUT /api/usuarios/<id-victima>
Authorization: Bearer <token-admin-A>
Content-Type: application/json

{"contrasena":"Control-atacante-123"}
```

Respuesta 200. Login local con esa clave y `organizacionId` de B devuelve el rol Administrador de B. La prueba de plataforma con víctima superadministradora también devuelve 200. No se modifica la bandera superadministradora: se toma su identidad.

Precondiciones: administrador legítimo de A, correo de una identidad activa y cupo para vínculo activo. La rama de membresía Inactivo no consume cupo, aunque la cadena probada usó una membresía activa. El bloqueo de creación de identidades nuevas U02 no impide vincular identidades existentes.

Remediación prioritaria: separar edición de identidad de administración de membresía, impedir cambio de credenciales globales por administradores de clientes, exigir aceptación autenticada/verificación del correo para vinculación y aislar cuentas de plataforma de administración por tenants. Una vez corregido, invalidar sesiones afectadas y revisar registros; no ejecutar esas acciones sobre producción desde este reproductor.

Controles correctos y límites

La búsqueda de membresías para GET/PUT/DELETE incluye `req.organizacionId`; body `organizacionId` no la reemplaza. Selector de organizaciones comprueba usuario+organización+Activo. `resolverFincaActiva` valida ID, lista de fincas y pertenencia a organización en cada petición; una finca de B no puede abrirse con membresía A.

No hay roles distintos por finca dentro de la misma organización: una membresía contiene un único rol y una lista de acceso. Un administrador con acceso restringido a fincas sigue teniendo administración de membresías de la organización; documentar esa semántica para no venderla como administración limitada por finca.

El usuario cuya organización activa sea suspendida no puede usar el selector con ese token porque `auth` rechaza antes del controlador; puede iniciar sesión de nuevo eligiendo otra organización activa. No se encontró endpoint de suplantación de plataforma. El superadministrador se asigna por script administrativo; sus rutas verifican bandera y estado global actuales. U01 elude este control tomando una cuenta que ya posee la bandera.

7. Cupos de usuarios, planes y concurrencia

Plan	Membresías activas máximas	Roles disponibles	Auditoría avanzada
Esencial	3	Los seis	No
Gestión	8	Los seis	No
Pro	20	Los seis	Sí
Premium	50	Los seis	Sí

Fuente: planes.js. Cuenta membresías Activo por organización, incluidas administradoras; no usuarios globales únicos entre clientes. Inactivos no consumen. Una membresía activa de una identidad global inactiva sigue contándose, pues el contador no une Usuario.

`asegurarPuedeCrearUsuario` comprueba estado de plan y rol; `ejecutarConReservaCuota` toma reserva por organización/recurso, vuelve a contar dentro de la exclusión y libera en finally. Índice único `(organizacionId,recurso)`, reserva de 15 minutos, hasta 80 reintentos de 25 ms. Altas y reactivaciones usan esta vía.

Prueba solicitada: no hay invitaciones de miembros para enviar concurrentemente. Se probaron sus equivalentes reales: dos POST de vinculación de cuentas existentes a Esencial con dos miembros activos. Se simularon 75 ms de latencia de persistencia para solapar las operaciones: **uno 201, otro 422 `PLAN_USUARIOS_LIMIT_REACHED`; tres miembros finales**. Si el segundo llega después del primer guardado, la prevalidación puede devolver 403 `PLAN_USER_LIMIT_REACHED`, antes de reservar. No se afirma haber certificado atomicidad de MongoDB a partir del adaptador.

No existe Pendiente en membresías; la invitación de aprovisionamiento ya cuenta como Activo. Un downgrade de Stripe actualiza código de plan sin desactivar usuarios. `sobreLimite.usuarios` informa exceso y nuevas altas/reactivaciones activas fallan; sesiones de miembros actuales permanecen operativas. Se reprodujo estado con cuatro activos en Esencial, exceso informado y autenticación permitida. Definir cómo resolver exceso sin bloquear al último administrador ni eliminar autoría.

8. Logout, offline y baja de miembros

Al logout, `App.js` consulta pendientes **del contexto actual**, pide confirmación si existen, llama `limpiarDatosOfflineContexto(sesion)` y elimina `ganaderiaSesion`. Esa limpieza filtra por clave `organizacion:finca:usuario`. La prueba creó inventario y colas en dos fincas del mismo usuario; después de salir desde una permanecieron los registros de la otra. Un fallo de limpieza se ignora con `.catch(() => {})` y la sesión se cierra igualmente.

En expiración 401, fallo de validación inicial o token offline vencido se elimina sesión sin limpiar IndexedDB. Los eventos de UI no incorporan borrado de todos los contextos del titular. Existe `limpiarTodosDatosOffline`, pero no se usa en esos caminos. `ganaderiaEspecie` y otras preferencias locales también pueden permanecer; no contienen el JWT.

Dispositivo compartido: el selector normal de datos incluye usuario además de finca/organización, por lo que el segundo usuario no recibe automáticamente la caché del primero. Sin embargo, los registros permanecen sin cifrar y accesibles a alguien con control del almacenamiento del mismo navegador. Esto es riesgo de retención local, no una fuga remota entre tenants demostrada. Al volver a entrar el mismo usuario en otra finca, su cola antigua puede seguir allí pese al mensaje de descarte del logout.

PWA: `vite.config.js` precachea recursos estáticos y no configura runtime cache API; `limpiarCacheApiLegado` borra `api-cache` al iniciar. Las peticiones JSON usan `cache:'no-store'`. La ruta de imágenes protegidas responde `private,max-age=300`, y no hay limpieza explícita del caché HTTP de imágenes al salir. No se verificó caché efectiva en un navegador real.

Duración offline: al arrancar se decodifica `exp` sin validar firma y se rechaza sesión vencida; el JWT emitido dura ocho horas. Después de arrancar no hay temporizador ni verificación de expiración en `guardarCambiosPendientes`; se pudo encolar una operación con token vencido. No se puede garantizar máximo de ocho horas con una pestaña ya abierta. Manipular el almacenamiento local no produce un JWT válido ante el servidor.

Revocación durante offline: el backend relee membresía/rol al sincronizar. Token sin membresía obtiene 403; la simulación del sincronizador conserva el error en español y no marca éxito ni modifica caché. Trata el 403 como reintento hasta cinco intentos, en vez

de exigir reautenticación inmediatamente. Un 401 elimina sesión antes de terminar la actualización de cola, por lo que merece una prueba de navegador adicional.

Bajas operativas: eliminación solo borra membresía; no borra Usuario, tareas, notificaciones, responsables IATF/tratamientos ni referencias históricas. Esto conserva autoría, pero no hay transferencia de pendientes, cancelación de asignaciones ni resumen de impacto previo. Los servicios de usuarios asignables sí excluyen miembros sin acceso; no resuelven por sí solos las asignaciones anteriores.

9. Auditoría administrativa y datos personales

Eventos y garantías

Evento	Cobertura real	Limitación
Login exitoso	POST, estado, IP, duración, correo en body, organización resuelta	No establece `req.usuario`; falta actor normalizado
Login fallido/limitado	POST y HTTP de error	Sin organización/actor; no aparece en consulta aislada del cliente
Alta, edición, rol, estado, baja, selector	Middleware genérico de mutaciones	Body solicitado, no valores anterior/nuevo persistidos
Invitación inicial y estado organización	Petición de plataforma, actor plataforma	Sin `req.organizacionId` del tenant destino; no es historia accesible al cliente
Forgot/reset	POST, secretos estándar ocultos	Sin actor resuelto; campos camelCase filtran, U09
Cambio plan Stripe	`EventoFacturacion` almacena evento/tipo/estado/organización	Webhook montado antes de auditoría genérica; sin snapshot antes/después
Eliminaciones	Método, ruta, actor de petición	No registra contenido eliminado ni versión previa
Exportaciones GET	**No auditadas** por este middleware	`METODOS_AUDITABLES` excluye GET
Logout	No hay petición al servidor	Sin registro ni revocación

`Auditoria` almacena quién si existe `req.usuario`, qué mediante método/ruta, timestamp, IP, userAgent y body sanitizado. No guarda un diff. `recursoId` depende de params disponibles al finalizar; no es un registro semántico estable. Es asíncrono después de responder y un fallo de escritura se registra en consola: no asegura durabilidad de cada acción sensible.

Consulta solo A y feature Pro/Premium. Hook probado fuerza organización actual aunque la consulta original especifique B. No hay rutas de edición/borrado de auditoría. **Eso no la vuelve inmutable en base de datos:** no hay almacenamiento append-only ni firma, y el modelo permite mutaciones. Con `required:false`, consultas sin contexto no se aíslan; las rutas de cliente sí pasan por auth/contexto.

U09: `normalizarClave` convierte a minúsculas, pero el Set contiene `confirmarContrasena`, `confirmarPassword` y `resetPasswordToken` con mayúsculas. La prueba HTTP persistió `confirmarContrasena` literal en el registro simulado. `contrasena`, `password` y `token` sí se ocultaron. Además se guarda `req.originalUrl` sin sanitizar; evitar secretos en URL y aplicar redacción también a ella.

Privacidad y configuración

U16 es una **brecha funcional verificable, no una certificación de incumplimiento jurídico**: perfil GET permite acceso propio; no existe autoservicio de corrección para no administradores, endpoint de supresión/anonimización, registro de aceptación/versionado de términos o aviso de privacidad en modelos/flujo inspeccionados. DELETE de usuario solo quita membresía, no elimina datos personales. No se verificaron procedimientos manuales externos al repositorio.

Como referencia del alcance de acceso, rectificación y supresión se consultó la Ley 8968 en el SCIJ y la publicación oficial de reforma del artículo 7 de agosto de 2025. Diseñar un procedimiento de atención y anonimización con criterios de conservación, preservando identificadores históricos cuando corresponda; la valoración jurídica requiere revisar finalidad y obligaciones concretas.

U17: CORS establece un origen mediante `FRONTEND_URL` o localhost; no refleja el origen atacante. `credentials:true` no implica que se estén usando cookies de sesión. La app no agrega Helmet, CSP, HSTS ni fuerza HTTPS; la prueba HTTP local recibió 200 sin esas

cabeceras. **No prueba que el despliegue sea HTTP inseguro**: no se inspeccionó terminación TLS/proxy. `frontend/vercel.json` solo contiene rewrite. No se encontraron respuestas explícitas con `error.stack` en los controladores revisados; varias devuelven `error.message` interno incluso en producción. Validar cabeceras y errores desplegados en staging.

Dependencias

Instalado y comprobado con `npm ls`: **bcrypt 6.0.0**, **Express 4.22.3**, **proxy-addr 2.0.8**. JWT usa crypto nativo; no hay jsonwebtoken/passport/argon2 instalados en este árbol consultado.

`npm audit --json` se ejecutó contra el registro npm: **6 paquetes con severidad high, 0 critical**: `brace-expansion`, `braces`, `chokidar`, `minimatch`, `nodemon`, `picomatch`. Son paquetes de desarrollo en este árbol; **`npm audit --omit=dev --json`: 0 vulnerabilidades**. No se reportó vulnerabilidad conocida de bcrypt en esa consulta. No equivale a prueba de ausencia de defectos propios ni cubre una librería JWT externa porque no se usa.

Ejemplos de avisos dev devueltos por npm: braces GHSA-vfj7-8cjw-p6xm, minimatch GHSA-23c5-xmqv-rm74, picomatch GHSA-c2c7-rcm5-vvqj. U18 se limita al resultado del escáner; no se ejecutó un DoS ni se demostró exposición de esos paquetes a peticiones de autenticación. No aplicar automáticamente la sugerencia de downgrade mayor de nodemon emitida por npm.

10. Plan de acción priorizado y reproducciones

Todas las referencias U01–U17 corresponden a etiquetas del reproductor; U18 se reproduce con los comandos de npm audit anteriores. Una misma comprobación puede demostrar más de un aspecto de un hallazgo.

ID	Prioridad	Hallazgo y evidencia reproducida	Acción y criterio de cierre
U01	**P0**	Vinculación sin aceptación + edición de contraseña global; POST→PUT→login B→plataforma, HTTP local 201/200	Separar identidad/membresía, consentimiento y cuentas privilegiadas; pruebas negativas completas de la cadena
U02	P1	Cuenta inexistente cae en `usuarioExistente?.estado !== 'Activo'`; alta válida 409	Corregir condición y validar alta nueva, rollback, política de clave y cuota
U03	P1	Único administrador degradado, desactivado o eliminado con 200	Invariante atómica de al menos un administrador activo; cubrir dos bajas concurrentes y globalmente inactivos
U04	P1	PUT acepta clave de un carácter; JWT antiguo funciona tras PUT/reset y obtiene nuevo JWT vía selector sin contraseña	Política uniforme; identidad propia verificada; versión/revocación de sesiones y tokens de reset previos, también al renovar contexto
U05	P1	Límites por par IP+correo eludidos variando una dimensión; reset admite correo arbitrario	Límites independientes por IP/cuenta/acción, almacenamiento compartido, claves apropiadas y pruebas de proxy
U06	P1	Cuenta inactiva devuelve 403 antes de contraseña; diferencia de costo bcrypt	Respuesta uniforme, hash señuelo para inexistentes, no revelar estado sin credencial válida
U07	P2	Acepta encabezado incompatible y sin exp si se firma con secreto de pruebas	Verificador estricto HS256, exp obligatorio tipado, issuer/audience y comparación segura; no tratarlo como `none` explotable sin firma
U08	P1	Dos resets concurrentes aceptan mismo token	Consumo atómico condicionado a hash y vigencia; transacción con versión de credenciales; solo un resultado exitoso
U09	P1	Sanitizador filtra confirmarContrasena/resetPasswordToken	Normalizar Set, lista permitida de metadatos, redacción de URL; prueba con campos anidados/camelCase
U10	P2	Login sin actor, export GET sin registro, sin antes/después	Eventos semánticos, actor/tenant resueltos, snapshots mínimos, durabilidad y consultas de plataforma

ID	Prioridad	Hallazgo y evidencia reproducida	Acción y criterio de cierre
U11	P1	Nombre HTML se inserta literal; sin RESEND_API_KEY log incluye enlace secreto	Escapar variables, evitar log de mensajes/tokens, impedir modo simulación en producción y comunicar fallo de entrega
U12	P1	Logout conserva datos y cola de otra finca; expiración no limpia	Limpiar todos los contextos del usuario y manejar fallo de borrado; probar dispositivo compartido y migraciones
U13	P1	Se encola después de expirar token; comprobación solo inicial	Reevaluar expiración durante uso/antes de lectura y cola; limitar vida offline y separar expiración de fallo de red
U14	P1	Router de archivos autoriza Consulta y no consulta finca de origen	Resolver documento propietario; validar `drone.ver`, tenant y finca antes de entregar; no confiar en secreto del filename
U15	P1	Botones/lecturas discrepantes; cinco DELETE 403, BandasPorcinas rompe carga de Encargado	Compartir permisos por acción, separar gestionar de eliminar, cargar selectores solo con permiso; pruebas de componentes por rol
U16	P2	Sin autoservicio completo, anonimización/consentimiento ni proceso de baja con pendientes	Definir flujos de derechos y offboarding; conservar autoría sin retener datos innecesarios; registrar versiones de aceptación
U17	P2	App local sin cabeceras de seguridad ni HTTPS obligatorio	Verificar edge/staging, configurar cabeceras y proxy seguros, respuestas de error mínimas; no inferir estado del despliegue
U18	P2	npm audit detecta seis paquetes dev high; producción 0	Actualizar árbol de desarrollo de forma compatible y repetir auditoría; no hay ataque de autenticación demostrado por estos avisos

Orden de implementación: cerrar U01 primero; después sesiones/reset, último administrador y redacción de secretos; luego límites, archivos y offline; por último consistencia de pantallas, evidencia administrativa y endurecimiento. Antes de dar por remediado el módulo, ejecutar las cadenas de ataque contra una instancia local/staging con MongoDB y un navegador real, sin usar cuentas de clientes.